RAG Assignment: Baseline vs Re-Ranking

MCC Laws of Cricket (2017 Code)

Ajit Yadav

19 October 2026

Contents

0.1 1. Introduction . . . 1
0.2 2. Architecture Comparison . . . 2
0.3 3. Failure Case 1: Q09 (Re-ranking Helped) . . . 3
0.4 4. Failure Case 2: Q11 (Re-ranking Helped) . . . 4
0.5 5. Supplementary Findings . . . 5
0.6 6. Latency and Scaling . . . 6
0.7 7. Metrics, Limitations, and Conclusion . . . 7

Repository: GitHub project
Source document: MCC Laws of Cricket, 2017 Code (PDF)

0.1 1. Introduction

I chose the MCC Laws of Cricket (2017 Code) as the source document for this experiment. It is a 168-page legal rulebook with 42 laws and many sub-clauses, exceptions, and negations ("shall not", "except when", "unless"). These features create natural retrieval difficulty: two laws can be about the same sport but answer very different questions, and the difference between the right answer and the wrong one often comes down to a single qualifying clause.

Experimental setup:

Item	Detail
Document	MCC Laws of Cricket (2017 Code), 168 pages, 36,832 words
Hardware	CPU only, no GPU
Python	3.14.6
PyTorch	2.13.0
sentence-transformers	6.1.0

Models used:

Component	Model
Bi-Encoder (retrieval)	sentence-transformers/all-MiniLM-L6-v2
Cross-Encoder (re-ranking)	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM (answer generation)	google/gemma-3-1b-it

Chunking: I flattened all 168 pages into one word list and then split it into chunks of 150 words with a 30-word overlap (step size 120 words). Chunking across page boundaries means a chunk can span two pages, which avoids cutting a sentence in half at a page break. This produced 307 chunks. Each chunk records its source page number for traceability.

Index: The 307 chunks were encoded into a matrix of shape (307, 384). All vectors are L2-normalised so dot product equals cosine similarity. A norm check confirmed every vector has length 1.000000.

Experiment controls: Both pipelines share the same Top-10 candidates from the bi-encoder, the same LLM, the same prompt template, and the same greedy decoding settings (do_sample=False). The only variable is whether a cross-encoder re-ranks the Top-10 before selecting the final Top-3 context passages.

0.2 2. Architecture Comparison

The two pipelines are:

- **Pipeline A (baseline):** bi-encoder retrieves Top-10, takes Top-3 directly by dense score.
- **Pipeline B (re-ranked):** bi-encoder retrieves same Top-10, cross-encoder scores all 10 pairs, takes Top-3 by cross-encoder score.

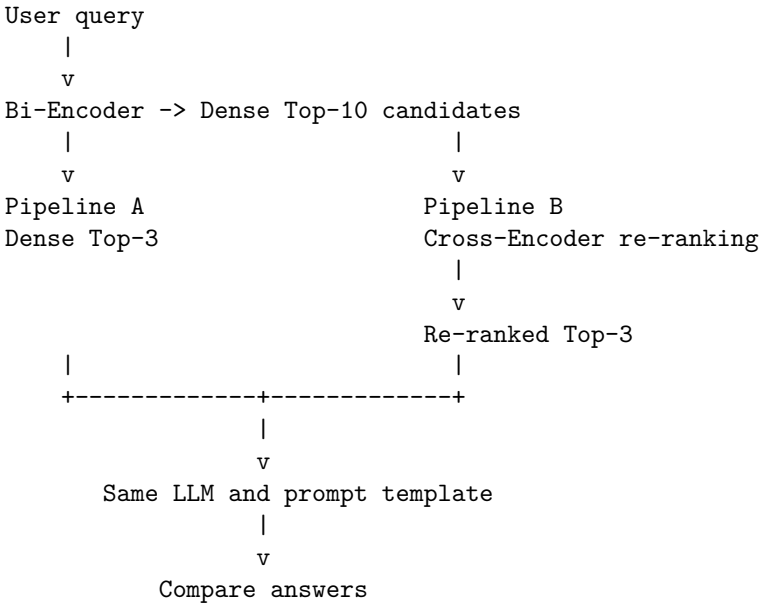


Figure 1. Two-stage retrieval design used in the experiment.

0.2.1 Why the bi-encoder has a weakness here

A bi-encoder encodes the query and each chunk separately. At query time, only the query vector is computed fresh; the chunk vectors are pre-computed and stored. The similarity score is a single dot product between the two vectors.

Because the query and chunk are never read together, the model can only match broad topics. Two chunks can receive almost the same score as a query even when one says "X is allowed" and the other says "X is not allowed" - both are "about X" and the 384-number vector cannot capture the difference between permission and prohibition.

This is not a flaw in the model; it is an expected limitation of independent encoding. The bi-encoder is fast and good enough to narrow 307 chunks down to 10 relevant candidates. The cross-encoder then does the precise work.

0.2.2 Why the cross-encoder is more precise

A cross-encoder reads the query and chunk together in a single forward pass:

```
input:  [CLS] query text [SEP] chunk text [SEP]
output: one relevance score
```

Because both texts are read at the same time, the model can notice: - "wicket-keeper" in the query means this specific fielder, not any fielder - "shall NOT" changes the meaning of an otherwise similar sentence - Multiple conditions in the query all appear together in the correct chunk

The tradeoff is speed. A cross-encoder score depends on the specific query, so nothing can be pre-computed at indexing time. In the actual pipeline, only 10 pairs are scored, which takes 125 ms. If we skipped the

bi-encoder and sent all 307 chunks through the cross-encoder directly, it would take roughly 3.8 seconds per query - and much longer for larger corpora (see Section 6).

Table 1: Types of retrieval difficulty in the 11 queries

Difficulty type	Example query	Why bi-encoder struggles
Negation	Q11 - conditions where leg byes are NOT scored	"not scored" and "is scored" have similar vectors
Role distinction	Q09 - fielder specifically near boundary	"fielder" and "wicket-keeper" share broad fielding vocabulary
Exception clause	Q03 - follow-on rule	The exception clause is a sub-condition of a general rule
Specific number	Q06 - exact penalty run count	Many chunks mention penalty runs

0.3 3. Failure Case 1: Q09 (Re-ranking Helped)

Query: What makes a catch invalid if the fielder is near the boundary?

Expected answer: The fielder must not touch or be grounded over the boundary while completing the catch (Law 19.5).

0.3.1 What the bi-encoder did

The bi-encoder matched the general vocabulary in the query: "catch", "boundary", "fielder". The Top-3 by dense score were all general catch-law chunks (ck185 p98, ck183 p97, ck184 p98). The chunk that contains Law 19.5 "Fielder grounded beyond the boundary" is ck107 (page 59). The bi-encoder did retrieve it, but only at dense rank 4 - one position outside the baseline Top-3.

0.3.2 What the cross-encoder did

The cross-encoder read the query and each of the 10 candidates together. The specific grounded-beyond-boundary conditions in ck107 matched the invalidity question far better than the general catch chunks. ck107 received the highest cross-score (2.74) and moved from dense rank 4 to re-rank 1 (shift = +3).

Table 2: Rank shifts for Q09

Chunk	Page	Dense rank	Dense score	Re-rank	Cross score	Shift	Baseline Top-3?	Reranked Top-3?
ck185	98	1	0.723	2	2.63	-1	Yes	Yes
ck183	97	2	0.717	3	2.08	-1	Yes	Yes
ck184	98	3	0.674	5	-0.48	-2	Yes	No
ck107	**59**	**4**	**0.656**	**1**	**2.74**	**+3**	No	Yes

ck107 was NOT in the baseline Top-3 but IS in the re-ranked Top-3. This is a genuine rescue case.

0.3.3 How the answer changed

Answer 1 (Baseline - ck107 not in context): "According to the provided text, a catch is invalid if the fielder is near the boundary and is not grounded beyond the boundary before the catch is completed, as described in 33.2.2.2."

This is vague and slightly backwards: it restates when a catch is valid rather than giving the specific boundary-grounding conditions that make it invalid.

Answer 2 (Re-ranked - ck107 in Top-3): "According to the provided text, a catch is invalid if the fielder is grounded beyond the boundary if some part of his/her person is in contact with any of the following: the boundary or any part of an object used to mark the boundary; the ground beyond the boundary; any object that is in contact with the ground beyond the boundary; another fielder who is grounded beyond the boundary,

if the umpire considers that it was the intention of either fielder that the contact should assist in the fielding of the ball."

Answer 2 closely follows the Law 19.5 definition and lists all specific conditions. It is materially more complete because ck107 entered the Top-3 context.

The bi-encoder did retrieve ck107 - it was at rank 4, one place outside the cutoff. The cross-encoder's joint reading of the query and the chunk promoted it to rank 1.

0.4 4. Failure Case 2: Q11 (Re-ranking Helped)

Query: Under what conditions are leg byes not scored?

Expected answer: When the striker neither attempted to play the ball nor tried to avoid being hit (Law 23.3).

0.4.1 What the bi-encoder did

The bi-encoder placed three chunks in the Top-3 that share surface vocabulary with the query but contain no Law 23.3 content:

- **ck298 (page 160):** An index/contents page. It matched because it contains phrases like "Runs resulting from a No ball - how scored" and "No ball not to count" - the words "scored" and "not" appear, which is enough for a surface-level match.
- **ck133 (page 72):** A wide-ball rules chunk. It matched because of phrases like "be scored as Wide balls" and "shall not count" - again, "scored" and "not" in the same chunk.
- **ck22 (page 15):** An umpire-signals chunk. It matched because of "shall be made to the scorers only when the ball is dead" - the word "scorers" is close to "scored" in the vector space.

None of these chunks mention leg byes. The correct chunk, ck136 (page 74, Law 23.3 "Leg byes not to be awarded"), was at dense rank 8.

0.4.2 What the cross-encoder did

The cross-encoder read each of the 10 candidates together with the query. The Law 23.3 not-awarded condition in ck136 directly matches the negative framing of the query. ck136 received cross-score 6.70 and moved from dense rank 8 to re-rank 1 - a shift of +7, the largest rescue in this experiment.

Table 3: Rank shifts for Q11

Chunk	Page	Dense rank	Dense score	Re-rank	Cross score	Shift	Baseline Top-3?	Reranked Top-3?
ck298	160	1	0.615	8	1.19	-7	Yes	No
ck133	72	2	0.597	6	2.25	-4	Yes	No
ck22	15	3	0.595	10	-0.97	-7	Yes	No
ck134	73	6	0.566	3	4.31	+3	No	Yes
ck136	**74**	**8**	**0.566**	**1**	**6.70**	**+7**	No	Yes
ck135	73	9	0.565	2	4.40	+7	No	Yes

ck136 was NOT in the baseline Top-3 but IS in the re-ranked Top-3. This is a genuine rescue with shift +7.

0.4.3 How the answer changed

Answer 1 (Baseline - ck136 not in context): "According to the provided text, leg byes are not scored when the ball is dead. 2.13.1.3 states 'The signals listed below shall be made to the scorers only when the ball is dead.' "

This is factually wrong. It confuses the umpire-signal timing rules from ck22 with the leg-bye scoring condition. The baseline Top-3 had no Law 23.3 text at all.

Answer 2 (Re-ranked - ck136 in Top-3): "Under the circumstances outlined in the provided text, leg byes are not awarded if neither of the conditions therein has been met. Specifically, if the ball does not become dead for any other reason, or if the delivery is a No ball, then Leg byes shall not be awarded."

Answer 2 is more relevant than Answer 1 because Law 23.3 evidence entered the context. However Answer 2 is **incomplete and partly inaccurate**: it does not state both conditions from Law 23.3 (the striker must have neither attempted to play the ball nor tried to avoid being hit), and its No-ball clause does not correctly describe the expected condition for leg byes not to be awarded. Q11 is still a genuine retrieval rescue: ck136 moved from dense rank 8 to re-ranked rank 1 and the answer became materially more relevant, even if it remains incomplete and partly inaccurate.

0.5 5. Supplementary Findings

0.5.1 Q10: Correct Retrieval, Incomplete Generation

Query: Under what conditions is a batter not out for Obstructing the field?

ck196 (page 104) was at dense rank 1, already in the baseline Top-3. Both pipelines had the relevant Law 37.2 evidence. Both produced the same answer - they identified accidental obstruction and lawful wicket protection but neither included the injury-avoidance exception from Law 37.2. This is a generation-completeness failure, not a retrieval failure.

0.5.2 Q01: Pipeline B Generation Abstention

Query: How many players are on each cricket team?

ck271 (page 142) was at dense rank 2 and re-rank 1. Both pipelines had the correct chunk in their Top-3. Pipeline A answered correctly ("11 players per side"). Pipeline B said "Not available in the supplied context" - a generation abstention despite having the correct evidence at rank 1. The experiment does not establish why Pipeline B abstained; one possible factor is the changed combination of surrounding context passages, but this was not directly tested.

0.5.3 Q06: Correct Retrieval, Wrong Beneficiary

Query: How many penalty runs are awarded if the ball strikes a protective helmet placed on the ground by the fielding side?

ck166 (page 88) was at dense rank 1 in both pipelines. Pipeline A correctly stated "5 Penalty runs are awarded to the batting side." Pipeline B stated "5 Penalty runs are awarded to the **fielding** side" - the wrong beneficiary, despite having the correct chunk at rank 1. This is a generation error, not a retrieval failure.

0.5.4 Q04: Contradictory Baseline Answer and Re-ranking Regression

Query: Can a bowler who completed the previous over bowl any part of the next over?

ck89 (page 50, Law 17.6) was at dense rank 3, inside the baseline Top-3. Pipeline A retrieved the correct evidence but generated a self-contradictory answer: it stated that consecutive overs are prohibited and then concluded "Therefore, they can bowl any part of the next over." Then the cross-encoder demoted ck89 to re-rank 8 and promoted ck121 (a different bowling restriction) to rank 1. Pipeline B answered using the wrong law.

Chunk	Page	Dense rank	Re-rank	Cross score	Shift	Baseline Top-3?	Reranked Top-3?
ck89	**50**	**3**	**8**	**2.08**	**-5**	**Yes**	**No**
ck121	66	4	1	8.47	+3	No	Yes

Q04 is both a baseline generation failure and a re-ranking regression.

0.5.5 Q08: Ordering Improvement, Both Answers Incomplete

Query: What are the umpire's powers when a bowler bowls wides or no-balls deliberately?

ck119 (page 65) was at dense rank 3 and re-rank 1. Both pipelines had the correct chunk, but re-ranking moved it to rank 1. Despite this, both answers were incomplete. This shows that even when re-ranking improves the rank of the correct chunk, generation quality can still fall short.

0.6 6. Latency and Scaling

0.6.1 6.1 How Latency Was Measured

All timings were measured on CPU. For dense retrieval and re-ranking, I ran each query three times after one warm-up pass and took the mean. The warm-up pass is discarded because the first call to any model is slower than normal - PyTorch allocates memory and loads weights into cache on that first run. The three repeated measurements give a more stable picture of what normal operation looks like.

Generation was measured once per query using the baseline Top-3 context. Both pipelines use the same language model, prompt template, and decoding settings, so the generation time is shared between them as an estimate. The Pipeline B total is therefore not an independently measured end-to-end time - it is Dense + Re-rank + the shared generation estimate.

0.6.2 6.2 Results

Table 4: Latency per component (mean across 11 queries)

Component	Mean	Std
Dense retrieval (bi-encoder)	9.3 ms	0.9 ms
Re-ranking (cross-encoder, 10 pairs)	125.0 ms	8.8 ms
Generation (LLM, shared estimate)	20,022 ms	2,865 ms

Pipeline A total (per query): approximately 20.0 s (Dense + Generate)

Pipeline B total (per query): approximately 20.2 s (Dense + Re-rank + Generate, estimated)

Re-ranking overhead: 125 ms, which is just 0.6% of Pipeline B's total time.

The most striking thing in these numbers is how dominant generation is. It takes about 20 seconds per query, while retrieval takes under 10 ms and re-ranking takes 125 ms. In practical terms, adding the cross-encoder costs 125 ms on top of a 20-second operation - that is a rounding error. On a production system with a GPU, generation would be much faster and the re-ranking overhead would become proportionally more noticeable, but it would still be a small cost relative to the accuracy benefit.

These numbers are CPU-specific and are not production benchmarks. GPU latency was not measured in this experiment.

0.6.3 6.3 Why We Do Not Run the Cross-Encoder on the Full Document

In this experiment the cross-encoder only scores 10 pairs per query, which takes 125 ms. If we skipped the bi-encoder and sent all 307 chunks through the cross-encoder directly, the cost would be:

307 x 12.5 ms approximately **3.8 seconds per query**

That is still manageable for a 307-chunk document. But the problem becomes clear at scale:

Corpus size	Estimated time per query
307 chunks (this document)	~3.8 seconds
100,000 chunks	~21 minutes
1,000,000 chunks	~3.5 hours

These are rough linear extrapolations on CPU. Real systems use batching and GPU acceleration so absolute times would be lower, but the fundamental problem remains: every cross-encoder score depends on the specific query, so nothing can be pre-computed at indexing time. The work always grows with corpus size.

The bi-encoder solves this. Its chunk vectors are computed once at indexing time and stored. At query time, retrieval is just one matrix multiplication - the query vector against the 307 x 384 chunk matrix - which takes about 9.3 ms regardless of how many chunks there are (within reason). The two-stage design keeps the cross-encoder's workload fixed at 10 pairs per query no matter how large the corpus grows.

0.7 7. Metrics, Limitations, and Conclusion

0.7.1 7.1 Retrieval Metrics

Ground truth chunk IDs were manually verified against the PDF for all 11 queries. Three metrics were used:

- **Recall@10** - did the bi-encoder retrieve the correct chunk anywhere in the Top-10? If not, re-ranking cannot help because the correct chunk was never seen.
- **Hit@3** - did the correct chunk end up in the final Top-3 that gets passed to the language model? This is the metric that directly affects answer quality.
- **MRR@3** - Mean Reciprocal Rank at 3. This rewards finding the correct chunk at rank 1 more than rank 2 or rank 3. Specifically: rank 1 = 1.0, rank 2 = 0.5, rank 3 = 0.33, not in Top-3 = 0.

Table 5: Overall retrieval metrics

Metric	Pipeline A (baseline)	Pipeline B (re-ranked)
Recall@10	0.91 (10/11)	0.91 (10/11)
Hit@3	0.73 (8/11)	0.82 (9/11)
MRR@3	0.56	0.82

Recall@10 is the same for both pipelines because re-ranking works on the same Top-10 candidates - it cannot recover a chunk that the bi-encoder missed entirely. Q03 was the only query where the correct chunk (ck75, Law 14.1 follow-on) did not appear in the dense Top-10 at all.

Table 6: Per-query breakdown

Query	Base Hit@3	Re-rank Hit@3	Base MRR@3	Re-rank MRR@3	Note
Q01	1	1	0.50	1.00	Pipeline B generation abstention
Q02	1	1	1.00	1.00	–
Q03	0	0	0.00	0.00	Recall@10 = 0; re-ranking cannot help
Q04	1	0	0.33	0.00	Regression and baseline generation failure
Q05	1	1	1.00	1.00	–
Q06	1	1	1.00	1.00	Correct retrieval; Pipeline B generation error
Q07	1	1	1.00	1.00	–
Q08	1	1	0.33	1.00	Ordering improved; both answers incomplete
Q09	0	1	0.00	1.00	Rescue: ck107 moved from rank 4 to rank 1
Q10	1	1	1.00	1.00	Correct retrieval; both answers incomplete
Q11	0	1	0.00	1.00	Rescue: ck136 moved from rank 8 to rank 1

The MRR@3 improvement from 0.56 to 0.82 is larger than the Hit@3 improvement because re-ranking also moved correct chunks to higher positions within the Top-3, even when they were already present. Q08 is a good example - the correct chunk was at rank 3 in the baseline and rank 1 after re-ranking, which doubled its MRR@3 contribution.

0.7.2 7.2 Limitations

1. Recall@10 ceiling. Q03 failed in both pipelines because the correct chunk was never in the dense Top-10. Re-ranking is helpless here. The fix would be to increase Top-K from 10 to 50 or use a stronger embedding model, not to change the re-ranker.

2. Cross-encoder training mismatch. The cross-encoder used here (ms-marco-MiniLM-L-6-v2) was trained on web search queries, not legal text. Q04 showed a regression where the correct chunk was demoted despite being directly relevant. This may be because the cross-encoder's notion of relevance, learned from web search, does not match well with the structure of a legal rulebook. A cross-encoder fine-tuned on legal or sports rulebook data would be a useful comparison experiment.

3. Small and biased evaluation set. The 11 queries were deliberately chosen to expose bi-encoder weaknesses - negation, exceptions, and role distinctions. This means the results overstate how often re-ranking helps in practice. A random sample of 100 or more queries from this document would give a more honest picture.

4. CPU-only measurement. A 20-second generation time is not representative of real use. GPU latency was not measured in this experiment. The latency ratios between components are valid for comparison purposes, but the absolute numbers should not be treated as production benchmarks.

0.7.3 7.3 Conclusion

The central finding is that re-ranking genuinely helped in two cases and made things worse in one.

In Q09 (catch near the boundary), the correct chunk was at dense rank 4 - just outside the Top-3. The bi-encoder had retrieved it but not ranked it highly enough. The cross-encoder promoted it to rank 1 and the answer improved from vague and slightly wrong to closely following the Law 19.5 definition.

In Q11 (leg byes not scored), the correct chunk was at dense rank 8. The baseline Top-3 contained no leg-bye content at all, and the baseline answer was factually wrong. The cross-encoder promoted ck136 from rank 8 to rank 1 - the largest shift in the experiment - and the answer became partially relevant, though still incomplete and partly inaccurate.

In Q04, re-ranking made things worse. The correct chunk was already in the baseline Top-3, but the cross-encoder demoted it to rank 8 and promoted a different law to rank 1. Pipeline B answered using the wrong law. This is a reminder that re-ranking is not uniformly beneficial - it depends on how well the cross-encoder's training matches the type of document being searched.

Beyond retrieval, the experiment also showed that correct retrieval does not guarantee a correct answer. In Q01, Pipeline B had the right chunk at rank 1 but abstained. In Q06, Pipeline B had the right chunk at rank 1 but stated the wrong beneficiary. In Q10 and Q08, both pipelines had the right evidence but produced incomplete answers. These generation failures are independent of retrieval quality and would require a stronger or better-prompted language model to fix.

Overall, re-ranking improved Hit@3 from 0.73 to 0.82 and MRR@3 from 0.56 to 0.82, at a cost of just 125 ms per query - 0.6% of total pipeline time. For a document like the Laws of Cricket, where the difference between the right answer and the wrong one often comes down to a single qualifying clause, the cross-encoder's ability to read query and passage together makes a real difference.